

BARTHÉLÉMY Romain
RAFFI Jean-Baptiste



Cahier des charges - Projet R&D

Responsable : OREIBY Ghassan
2025-2026

Sommaire

Sommaire	2
1. Contexte du projet	3
2. Objectifs du projet	4
3. Périmètre fonctionnel	5
4. Architecture technique	6
5. Données manipulées	8
6. Contraintes et exigences	9
7. Livrables attendus	10
8. Compétences mobilisées	11
9. Compétences acquises	12

1. Contexte du projet

Dans les infrastructures cloud et les environnements virtualisés, des milliers de services d'applications s'exécutent simultanément.

La consommation de ressources (CPU, mémoire, réseau, I/O) varie considérablement selon les périodes : pics d'activité, heures creuses, saisonnalité ou événements spécifiques.

Une allocation statique des ressources peut donc mener à deux problèmes majeurs :

- Sous-allocation : dégradation des performances, augmentation du temps de réponse et non-respect des SLA (Service Level Agreement).
- Sur-allocation : gaspillage de ressources et coûts opérationnels excessifs.

L'enjeu est donc de rendre la gestion des ressources intelligente, dynamique et anticipative. Ce projet vise à développer une application web prédictive capable de planifier, réserver et libérer les ressources serveur selon la charge estimée à venir.

2. Objectifs du projet

Objectif général

Développer une plateforme web permettant d'anticiper et d'automatiser la réservation de ressources serveur, en s'appuyant sur des modèles de Machine Learning pour prévoir la charge et ajuster les allocations en conséquence.

Objectifs spécifiques

1. Collecte et analyse des données

- Récupérer les métriques d'utilisation des serveurs (CPU, mémoire, réseaux, stockage...).
- Construire un jeu de données historique (logs, métriques temps réel).

2. Prédiction de la charge serveur

- Entraîner un modèle de prévision (régression, séries temporelles, ou ML supervisé) pour estimer la charge future.
- Calculer la charge moyenne prévue par serveur et détecter les pics.

3. Réservation automatisée

- Développer un module capable de réserver dynamiquement des conteneurs (pods) en fonction des prévisions.
- Intégrer un système de gestion des clusters (Kubernetes, k8s).

4. Libération proactive

- Identifier les tâches terminées ou les périodes creuses.
- Libérer automatiquement les ressources et les réallouer aux serveurs ayant une demande croissante.

5. Interface utilisateur (Dashboard & API)

- Créer une interface web intuitive permettant de :
 - Visualiser la charge actuelle et prédite
 - Surveiller la charge actuelle et prédite
 - Gérer manuellement les réservations en cas de besoin
 - Générer des alertes en cas de dépassement de seuil critique

6. Automatisation & supervision

- Définir des seuils d'alerte automatique (CPU > 90%, mémoire saturée...).
- Permettre aux administrateurs d'intervenir manuellement via l'interface.

3. Périmètre fonctionnel

Fonctionnalité	Description	Priorité
Collecte des métriques	Extraction automatique des logs et données de performance serveur	Haute
Prédiction de charge	Entraînement de modèle ML pour estimer la charge future	Haute
Réservation automatique	Allocation dynamique des conteneurs selon la charge prévue	Haute
Libération automatique	Désallocation des ressources inutilisées	Moyenne
Dashboard web	Visualisation de la charge, des prévisions et alerte	Haute
API REST	Interface de communication entre le moteur ML et le système d'orchestration	Moyenne
Alertes manuelles	Intervention et ajustement par un administrateur	Moyenne

4. Architecture technique

Frontend (interface utilisateur) :

- **Framework** : Next.js (React 19, App Router)
- **Langage** : TypeScript / JavaScript
- **Interface** :
 - Tableau de bord affichant la charge serveur actuelle et prédite (graphiques, jauges, logs en temps réel)
 - Visualisation interactive des clusters et conteneurs disponibles
 - Système d'alertes visuelles en cas de surcharge prévue
- **Librairie UI** :
 - TailwindCSS pour le style et la mise en page responsive
 - Recharts ou Chart.js pour les graphiques
 - Lucide-react / shadcn-ui pour les composants modernes
- **Rôle** : afficher les données récupérées du backend et permettre à l'administrateur d'intervenir (réservations manuelles, ajustements de seuils, etc.)

Backend (API et logique serveur)

- **API Routes de Next.js** : utilisées pour créer des endpoints REST internes, sans service externe supplémentaire.
- **Fonctionnalités principales** :
 - Collecte et stockage des métriques serveurs (simulées ou issues d'une base de données)
 - Communication avec le module de prédiction (ML)
 - Calcul en renvoi des prévisions au frontend
 - Gestion des réservations automatiques/libérations de ressources
 - Gestion des alertes et logs
- **Langage** : JavaScript / TypeScript
- **Base de données** : PostgreSQL (via Prisma ORM)
- **Sécurité** : authentification par token (JWT) et middleware de validation d'accès

Machine Learning / Prédiction

- Les modèles de prévision seront exécutés via :
 - Un service Python isolé (FastAPI ou Flask) exposé sur une API locale
 - Ou via Ollama si le modèle est localement hébergé
- Next.js communique avec ce module pour récupérer les prédictions temps réel.

Infrastructure et déploiement

- **Conteneurisation** : Docker
- **Orchestration** : Kubernetes (k8s) pour la gestion des services (API Next.js, base de données, module ML)
- **CI/CD** : GitHub ou GitLab pour automatiser les tests et déploiements
- **Hébergement** :

- Serveur cloud (ex: AWS, GCP, ou Render)
- Base de données gérée (Supabase / PlanetScale / Mongo Atlas)

5. Données manipulées

Type de données	Source	Utilisation
Logs système	Serveurs simulés ou réels	Extraction des métriques (CPU, RAM, I/O)
Historique de charge	Monitoring interne	Entraînement des modèles prédictifs
Prédictions	Module ML	Anticipation de la charge et allocation
Etat des clusters	Kubernetes API	Visualisation et supervision en temps réel

6. Contraintes et exigences

- **Performance** : traitement en quasi temps réel des données de charge
- **Scalabilité** : architecture modulaire et distribuée (micro-services)
- **Fiabilité** : tolérance aux pannes et gestion des erreurs d'allocation
- **Sécurité** : authentification des administrateurs, contrôle d'accès
- **Interopérabilité** : compatibilité avec différents environnements cloud (AWS, GCP, Azure, etc.)

7. Livrables attendus

1. Prototype fonctionnel de l'application web (code complet, conteneurs Docker, pipeline CI/CD)
2. Rapport technique décrivant :
 - L'architecture logicielle et réseau
 - Les algorithmes de prédiction utilisés
 - Les scénarios de test et résultats obtenus
3. Démonstration du dashboard présentant la prédiction et l'allocation dynamique des ressources

8. Compétences mobilisées

- Programmation web (React, TypeScript, Next.js, API REST)
- Systèmes & réseaux (Kubernetes, conteneurisation, orchestration)
- Machine Learning / Data Science (prévision de charge)
- DevOps (CI/CD, supervision, scalabilité)

9. Compétences acquises

- Ingénierie logicielle distribuée
- Data science appliqué à la supervision
- Scalabilité et automatisation cloud
- Conception d'interfaces intelligentes pour le monitoring